Technical Architecture & Operations Report for High-Traffic E-Commerce Platform

This report presents a comprehensive technical and operational assessment for implementing a high-traffic e-commerce platform capable of supporting significant user volumes during peak events such as Black Friday. The proposed solution addresses requirements for a resilient, scalable system supporting one million daily active users with peak concurrency of 250,000 users and 10,000 transactions per second.

by Mphasis Architect Team

Document Classification

Commercial In Confidence

Prepared for: IT Technical Lead - Blink & Buy

Date: 21 April 2025

Version: 1.0

Executive Summary

This report presents a comprehensive technical and operational assessment for implementing a high-traffic e-commerce platform capable of supporting significant user volumes during peak events such as Black Friday. The proposed solution addresses requirements for a resilient, scalable system supporting one million daily active users with peak concurrency of 250,000 users and 10,000 transactions per second.

The architecture employs a microservices approach with strategic redundancy, auto-scaling capabilities, and comprehensive monitoring. This report analyses the system components, information exchange patterns, potential failure points, and provides detailed mitigation and contingency strategies. Special attention is given to the distinction between average and peak load behaviour.

Our proposal delivers a solution that meets the stringent 99.99% uptime requirement while maintaining sub-200ms API response times and over 98% payment success rates, all within appropriate regulatory frameworks including PCI DSS, GDPR, and CCPA compliance.

Introduction

In today's digital marketplace, an e-commerce platform's ability to handle traffic surges directly impacts revenue, customer satisfaction, and brand reputation. This document outlines a detailed technical and operational plan to support the launch of a scalable, resilient e-commerce platform aimed at delivering seamless user experiences during high-traffic events such as Black Friday.

The focus is on aligning system design with business expectations, performance thresholds, regulatory compliance, and failure resilience.

Business Requirements & Technical Goals

Business Requirements

- Omnichannel Experience: Seamless shopping experience across platforms (Web, iOS, Android)
- Resilient Backend: Support for up to 1 million Daily Active Users (DAU)
- High Transaction Volume: 10,000 Transactions Per Second (TPS) during peak
- Payment Flexibility: Support for multiple payment gateways with failover
- Compliance: Adherence to GDPR, CCPA, and PCI DSS

Technical Goals

- Availability: 99.99% uptime (equating to approximately 52 minutes of downtime per year)
- Performance: API response time < 200ms
- Transaction Success: Payment success rate > 98%
- Observability: Real-time monitoring capabilities
- Security: Robust security controls and compliance with regulatory frameworks

System Architecture Overview

The proposed architecture follows a distributed microservices pattern deployed on cloud infrastructure, ensuring separation of concerns, independent scaling, and fault isolation.

Global Infrastructure Layer

- Multi-region cloud deployment with active-active configuration
- Strategic CDN distribution for static content
- DDoS protection and Web Application Firewall (WAF) services
- Global load balancing with intelligent routing

Platform Ecosystem

The platform consists of four primary domains:

1. Client-Facing Services: Manages user interfaces and API endpoints
2. Business Logic Services: Handles core e-commerce functionality
3. Data Management Services: Coordinates data persistence and retrieval
4. Operational Services: Provides monitoring, logging, and supporting functions

Integration Layer

- API Gateway
- Service Mesh
- Message Broker (Kafka, SQS)
- External system integrations (payment processors, logistics providers)

Component Architecture: Client-Facing Services

Edge Service Layer

- API Gateway with rate limiting and request throttling
- CDN integration for static content delivery
- Bot detection and prevention systems

User Interface Services

- Responsive web frontend (React)
- Native mobile apps (iOS/Swift, Android/Kotlin)
- Progressive Web App (PWA) support services

Component Architecture: Business Logic Services

User Management Domain

- Auth Service (SSO, MFA)
- Profile management service
- Preference and personalisation service

Catalogue Domain

- Product Service
- Search and indexing service
- Recommendation engine

Cart and Order Domain

- Cart Service
- Order & Fulfilment Service
- Pricing and promotion service

Payment Domain

- Payment Gateway Abstraction Layer
- Payment orchestration service
- Fraud detection service

Post-Purchase Domain

- Review & Ratings Service
- Notification Service (Email/SMS/Push)
- Order tracking service

Component Architecture: Data Management & Operational Services

Data Storage Layer

- RDS/PostgreSQL for relational data
- DynamoDB for session/catalog
- Redis/Memcached for caching
- Search index database (Elasticsearch)

Data Processing Layer

- Event streaming platform
- ETL services for analytics
- Data synchronisation service

Monitoring and Observability

- Real-time monitoring (Datadog, Prometheus)
- Centralised logging (ELK Stack)
- Alerting and notification service
- Distributed tracing system

Security Services

- Authentication and authorisation services
- Encryption services
- Secrets management

Infrastructure Management

- Auto-scaling controllers
- Configuration management
- Deployment orchestration
- Admin Dashboard (Analytics, ABAC)

Data Flow & Information Exchange Analysis

The platform supports various information exchange patterns with distinct characteristics:

User Interface Requests

Volume: 10-50 requests per user session

Frequency: Continuous during user sessions

Size: Small to medium (5-100KB per request)

Criticality: High (direct impact on user experience)

Product Data Queries

Volume: 40-60% of total system traffic

Frequency: Continuous, with spikes during promotions

Size: Medium (50-200KB per response)

Criticality: Medium-high (affects browsing experience)

Cart Operations

Volume: 20-30% of total system traffic

Frequency: Intermittent during shopping sessions

Size: Small (1-10KB per operation)

Criticality: High (directly impacts conversion)

Payment Transactions

Volume: Peak of 10,000 TPS during high-traffic events

Frequency: Concentrated during checkout activities

Size: Small (1-5KB per transaction)

Criticality: Critical (revenue-generating operations)

Order Management Events

Volume: Proportional to successful transactions

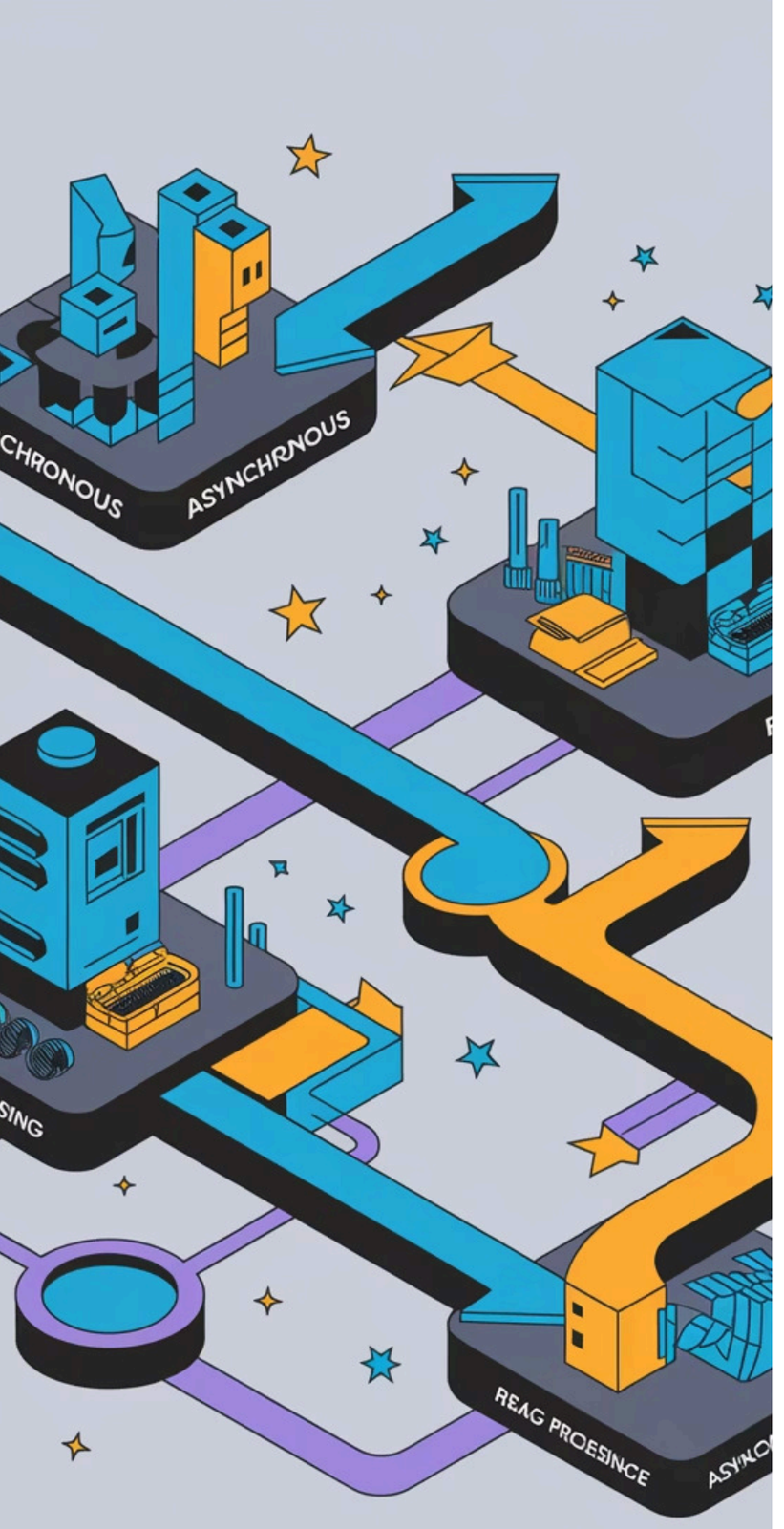
Frequency: Burst pattern following checkout peaks

Size: Small to medium (1-50KB per event)

Criticality: Medium (post-purchase experience)

System Interaction Table

System Interaction	Type of Data	Frequency	Volume	Speed Requirement
Frontend ↔️ Auth Service	Login/session data	Per session	Medium	< 100ms
Cart ↔️ Product Service	Product details/price	High (browsing)	Medium-high	< 150ms
Checkout ↔️ Payment Gateway	Payment token + confirmation	Per transaction	High	< 200ms
Orders ↔️ Fulfilment APIs	Order status, logistics	Asynchronous	Medium	N/A
Notifications ↔️ User	Email/SMS/Push	Event-driven	Low	Near real-time



Communication Patterns

Synchronous Request-Response

- Primary pattern for user-initiated operations
- Critical path operations requiring immediate results
- Utilised for most API interactions

Asynchronous Event-Driven

- Used for operation decoupling
- Order processing workflow
- Notification dispatching
- Analytics data collection

Bulk Data Processing

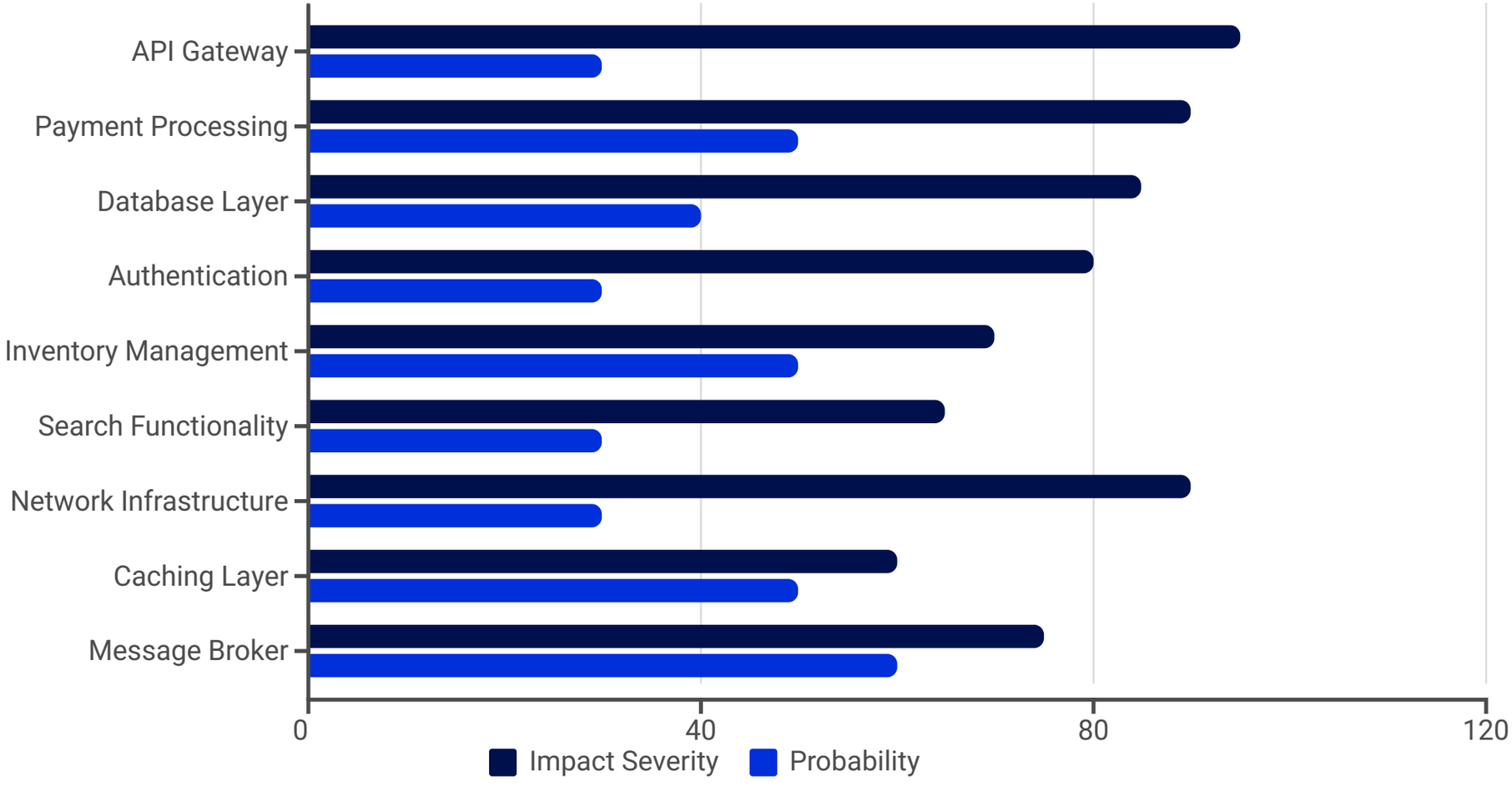
- Catalogue updates
- Pricing adjustments
- Recommendation model updates
- Reporting and analytics generation

Real-time Streaming

- Inventory updates
- Price change propagation
- User activity monitoring

Failure Mode Analysis

Critical System Failure Points:



The chart shows the impact severity (0-100) and probability (0-100) of different failure points in the system. Message brokers have the highest probability of failure during peak loads, while API Gateway and Network Infrastructure failures would have the most severe impact.

Resilience & Mitigation Strategies: Preventative Measures

Infrastructure Redundancy

- Multi-region deployment with active-active configuration
- Redundant instances of critical services
- Database clustering with automatic failover
- Multi-AZ deployment within each region

Traffic Management

- Implement circuit breakers at API Gateway
- Rate limiting and request prioritisation during high traffic
- Circuit breakers for failing downstream services
- Request queuing for non-critical operations

Data Protection

- Real-time database replication
- Point-in-time recovery capabilities
- Regular backup procedures with validation
- Data corruption detection mechanisms

Application Resilience

- Stateless service design where possible
- Idempotent API design
- Retry mechanisms with exponential backoff
- Request validation and sanitisation

Operational Readiness

- Comprehensive monitoring with alerting
- Runbooks for common failure scenarios
- On-call rotation with escalation paths
- Regular disaster recovery testing

Resilience & Mitigation Strategies: Contingency Plans

Service Degradation Strategy

- Feature toggling to disable non-essential features (e.g., reviews)
- Static content fallback for dynamic product pages
- Simplified search experience during index issues
- Read-only mode during write-critical outages

Payment Processing Contingencies

- Automatic payment gateway failover
- Multiple payment gateways with failover logic
- Asynchronous payment processing for peak loads
- Temporary order queueing with user notification

Data Recovery Procedures

- Database failover automation
- Cache rebuild procedures
- Search index reconstruction
- Data consistency verification protocols

Regional Failover Plan

- Traffic rerouting to healthy regions
- Geo-Failover: DNS-based redirection to alternate regions
- Data synchronisation between regions
- Capacity planning for single-region operation

Communication Plans

- User notification templates for service issues
- Stakeholder communication procedures
- Status page with real-time updates
- Post-incident review process

Pre-Event Load Testing

- Simulate 1.5x expected traffic using tools like Gatling/JMeter
- Identify and address bottlenecks before peak events

Load Management Strategy

Average Load Characteristics

Under average conditions, the system expects:

- 100,000-200,000 concurrent users
- 2,000-3,000 transactions per second
- 70% read operations, 30% write operations
- Predictable traffic patterns with minor fluctuations

The architecture for average load prioritises:

- Cost efficiency while maintaining performance
- Balanced resource utilisation
- Normal redundancy for standard resilience
- Regular maintenance windows
- On-demand instance scaling optimised for cost-efficiency

Peak Load Characteristics

During peak events, the system anticipates:

- Up to 250,000 concurrent users
- 10,000 transactions per second
- 80% read operations, 20% write operations
- Rapid traffic increases within short timeframes

The peak load architecture incorporates:

- Predictive auto-scaling before expected events
- Pre-warming auto-scaling groups
- Read-replica scaling for database offloading
- Cache warming and increased cache allocation
- Prioritisation of critical transaction paths
- Temporary relaxation of non-critical consistency requirements
- Provisioning based on traffic forecasts

Architectural Considerations for Variable Load

Elastic Scaling Design

- Horizontal scaling for stateless services
- Vertical scaling for database instances
- Container orchestration with auto-scaling groups
- Serverless components for highly variable workloads

Resource Allocation Strategy

- Base capacity sized for average load plus 50%
- Auto-scaling configured for rapid response
- Pre-warming procedures before known peak events
- Reserved instances for predictable base load
- On-demand resources for peak handling

Performance Optimisations

- Strategic caching with tiered approach
- Read/write splitting for database operations
- Use Redis for aggressive read caching
- Separate read/write DBs with replication
- Background job throttling during peak periods
- Asynchronous processing of non-critical operations

Implementation Considerations: Technology Stack

Infrastructure Platform

- Primary: AWS (leveraging multi-region capabilities)
- Alternative: Microsoft Azure with geo-redundancy

Application Technologies

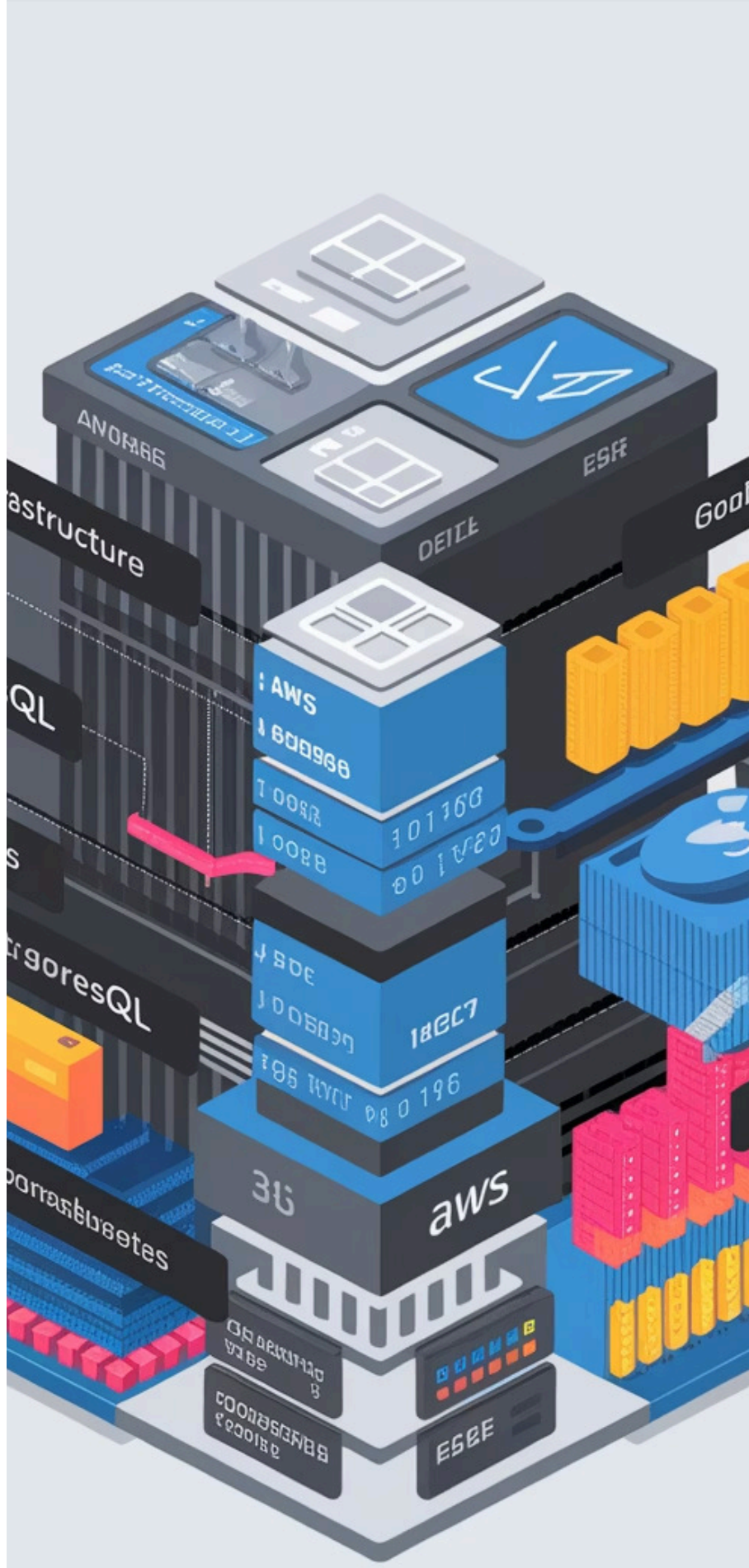
- Containerisation: Kubernetes for orchestration
- Backend Services: Java/Spring Boot microservices
- Frontend: React with server-side rendering capabilities

Data Technologies

- Primary Database: PostgreSQL with read replicas
- Document Store: MongoDB or DynamoDB for product data
- Caching Layer: Redis clusters
- Search: Elasticsearch with redundant indices
- Message Broker: Apache Kafka for event streaming

Operational Stack

- Monitoring & Alerting: Prometheus with Grafana dashboards or Datadog
- Logging: ELK stack (Elasticsearch, Logstash, Kibana)
- Tracing: Jaeger for distributed tracing



Cost Optimisation Strategies

Infrastructure Optimisation

- Right-sizing of compute resources
- Reserved instances for baseline capacity
- Spot instances for background processing
- Auto-scaling with appropriate thresholds

Storage Cost Management

- Tiered storage strategy
- Data lifecycle management
- Compression for applicable data
- Storage class transitions

Operational Cost Reduction

- Automated management and orchestration
- Comprehensive monitoring to prevent issues
- Self-healing system design
- Automation of routine tasks

Traffic Optimisation

- CDN integration for static content delivery
- Intelligent caching policies
- Response compression
- Efficient API design

Conclusion & Recommendations

The proposed architecture delivers a resilient, scalable e-commerce platform capable of handling the required load characteristics while maintaining high availability and performance. The solution balances technical excellence with commercial pragmatism, providing a cost-effective approach to meeting the challenging requirements.

Adopt a Multi-Region Strategy

The geographic redundancy provides essential resilience against regional failures and optimises performance for distributed users.

Implement Progressive Scaling

The distinction between average and peak load necessitates a dynamic scaling strategy that aligns resource allocation with actual demand.

Prioritise Payment Processing Resilience

As the most critical revenue-generating function, the payment processing pipeline warrants additional redundancy and failover capabilities.

Establish Comprehensive Monitoring

Early detection of potential issues is essential for maintaining the 99.99% uptime requirement.

Regular Resilience Testing

Chaos engineering practices should be implemented to regularly test failure scenarios and verify mitigation effectiveness.

Consider Future Growth

The architecture should be designed with capacity for at least 50% growth beyond current requirements without significant redesign.

With proper implementation of the recommended architecture and strategies, the e-commerce platform will be well-positioned to handle the demands of high-traffic events while providing an excellent customer experience and protecting revenue generation.